Guía de estudio: Scheduler de Linux y KGDB

Oct 20, 2026 · @Jose

Mapa de la exposición

La idea central: **lo que vimos en SO (PCB, estados, colas de listos, políticas) existe en Linux con nombres concretos, y con KGDB lo podemos ver en memoria mientras el kernel corre.** Estructura sugerida para unos 20 minutos:

1. **Puente con la teoría** (2 min): PCB → `task_struct`, cola de listos → `struct rq`, despachador → `__schedule()`.
2. **Procesos en Linux** (3 min): todo es una *task*; `fork` / `clone`; estados.
3. **Arquitectura del scheduler** (4 min): clases de planificación y cómo se elige la siguiente task.
4. **EEVDF** (5 min): el algoritmo actual del caso normal, con un ejemplo numérico de `nice`.
5. **KGDB** (2 min): qué es y cómo se conecta.
6. **Demo** (4 min): VM con QEMU, breakpoints en el scheduler y el experimento de `nice`.

Ten capturas de pantalla de cada paso de la demo por si algo falla en vivo.

Procesos en Linux

En Linux el PCB se llama **`struct task_struct`** (definido en `include/linux/sched.h`). Linux no distingue entre proceso e hilo: ambos son *tasks*. Un hilo es una task que comparte memoria, archivos y señales con otra.

Concepto de SO	En Linux	Dónde mirarlo
PCB	`struct task_struct`	`include/linux/sched.h`
Identificador	`pid` (hilo) y `tgid` (proceso; lo que muestra `getpid()`)	`task->pid`, `task->tgid`
Estado	`__state`	`task->__state`
Prioridad	`prio`, `static_prio`, `normal_prio`, `rt_priority`	`/proc/<pid>/sched`

Concepto de SO	En Linux	Dónde mirarlo
Política	`policy` (SCHED_NORMAL, SCHED_FIFO…)	`chrt -p <pid>`
Datos del planificador	`se` (`struct sched_entity`): `vruntime` , `deadline` , peso	`/proc/<pid>/sched`
Espacio de memoria	`mm` (`struct mm_struct`); `NULL` en hilos del kernel	`/proc/<pid>/maps`
Nombre	`comm` (16 caracteres)	`ps -o comm`
Lista de procesos	lista doblemente enlazada `tasks` , árbol `parent` / `children`	`pstree`

Estados (equivalencia con el diagrama de clase):

- `TASK_RUNNING` : cubre **listo y en ejecución** a la vez. Linux no los separa; la diferencia es si la task es `rq->curr` o está en la cola. Punto fácil de preguntar.
- `TASK_INTERRUPTIBLE` (S en `ps`): bloqueado, lo despierta un evento o una señal.
- `TASK_UNINTERRUPTIBLE` (D en `ps`): bloqueado esperando E/S; las señales no lo despiertan.
- `__TASK_STOPPED` (T) y `__TASK_TRACED` (t): detenido por SIGSTOP o por un depurador.
- `EXIT_ZOMBIE` (Z) y `EXIT_DEAD` (X), guardados en `exit_state` : terminó y espera que el padre haga `wait()` .

Creación: `fork()` , `vfork()` , `clone()` y `clone3()` terminan en `kernel_clone()` → `copy_process()` , que duplica la `task_struct` y llama a `sched_fork()` para inicializar los datos del scheduler. Después, `wake_up_new_task()` la mete en la runqueue. La memoria se copia con **copy-on-write**: las páginas se comparten hasta que alguien escribe.

Hilos del kernel: tasks sin espacio de usuario (`mm == NULL`), por ejemplo `kthreadd` (PID 2) o `ksoftirqd/N` . En `ps` aparecen entre corchetes.

Arquitectura del scheduler

El scheduler de Linux no es un solo algoritmo: son **clases de planificación** ordenadas por prioridad, y cada una implementa su propio algoritmo. Siempre gana la primera clase que tenga una task lista.

Orden	Clase (`struct sched_class`)	Políticas	Algoritmo	Uso típico
1	`stop`	(interna)	ninguno, gana siempre	migrar tasks entre CPUs, hotplug
2	`dl`	`SCHED_DEADLINE`	EDF + CBS (runtime, deadline, period)	tiempo real con garantías
3	`rt`	`SCHED_FIFO` , `SCHED_RR`	prioridad fija 1–99; RR con quantum de 100 ms	audio, control industrial
4	`fair`	`SCHED_NORMAL` , `SCHED_BATCH` , `SCHED_IDLE`	**EEVDF** (antes CFS)	casi todo: shell, navegador, compilaciones
5	`ext`	`SCHED_EXT`	el que cargues en BPF (sched_ext, desde 6.12)	experimentar con schedulers
6	`idle`	(interna)	corre la task idle del CPU	cuando no hay nada más

Prioridades internas: el kernel usa un solo número `prio` de 0 a 139, donde menor es más prioritario. De 0 a 99 son tiempo real; de 100 a 139 son tasks normales, y corresponden a `nice` de −20 a +19 (`prio = 120 + nice`).

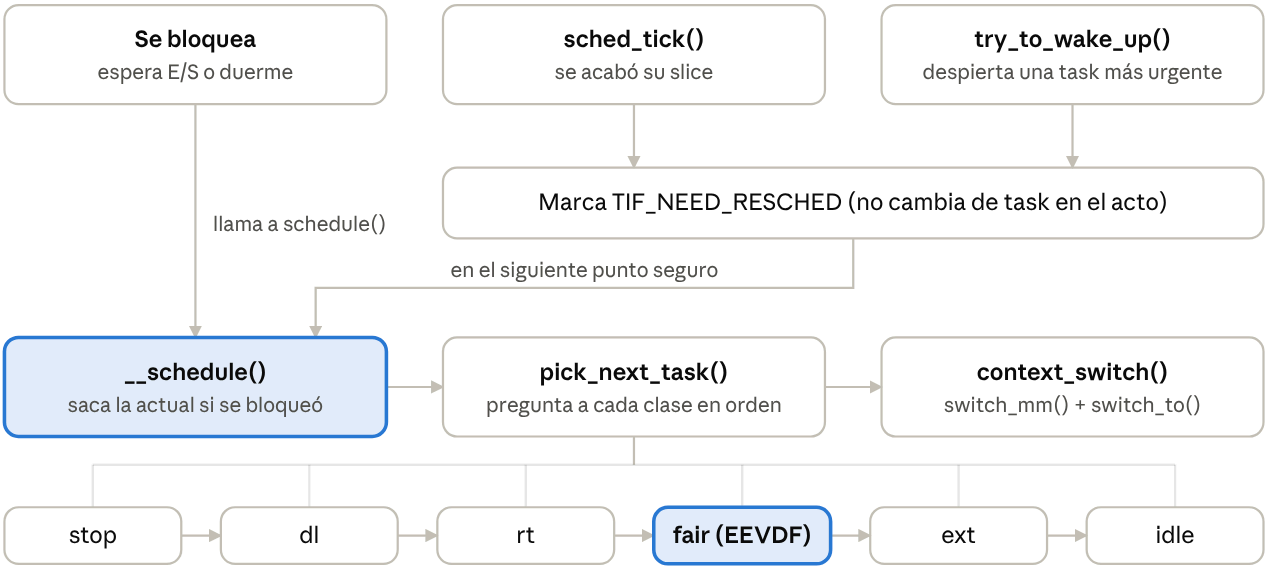
Una runqueue por CPU: cada CPU tiene su `struct rq` con una subcola por clase: `rq->cfs` (fair), `rq->rt` , `rq->dl` . También guarda `rq->curr` (la task en ejecución) y `rq->idle` . Esto evita un candado global: cada CPU decide sola.

El ciclo de planificación, en las funciones de `kernel/sched/core.c` :

1. **Algo pide replanificar.** Una task se bloquea y llama a `schedule()` (voluntario), o el kernel marca la bandera `TIF_NEED_RESCHED` porque se acabó su turno (`sched_tick()` , cada tick del timer) o despertó una task más urgente (`try_to_wake_up()`).
2. **`__schedule()`** se ejecuta: si la task actual ya no está en `TASK_RUNNING` , la saca de la cola (`dequeue_task`).

3. **`pick_next_task()`** recorre las clases en orden y pide a cada una su candidata (`pick_task_fair()`, `pick_task_rt()`…). Si todas las tasks son normales, salta directo a la clase fair.
4. **`context_switch()`** cambia el espacio de memoria (`switch_mm`) y los registros y la pila del kernel (`switch_to`). La nueva task continúa donde se había quedado.

Tres eventos llevan a __schedule(), que consulta las clases en orden



Gana la primera clase con una task lista. Las tasks normales están en fair.

ciclo de planificación · 3 eventos, 3 pasos, 6 clases

Bloquearse llama a `schedule()` directamente; el tick y los despertares solo marcan la bandera, y el cambio ocurre en el siguiente punto seguro.

Despertar: `try_to_wake_up()` elige CPU (`select_task_rq`), pone la task en `TASK_RUNNING`, la encola (`enqueue_task`) y, si es más urgente que la actual, marca `TIF_NEED_RESCHED` (`wakeup_preempt`).

Protección contra el hambre: una task RT con un bucle infinito podría congelar el sistema. Por eso se reserva tiempo para las tasks normales: antes con *RT throttling* (95% de cada segundo para RT, `/proc/sys/kernel/sched_rt_runtime_us`) y en kernels recientes con un *deadline server* para la clase fair.

De CFS a EEVDF

Desde el kernel **6.6 (2023)** la clase fair usa **EEVDF** (*Earliest Eligible Virtual Deadline First*) en lugar de CFS. Ambos buscan repartir la CPU en proporción al peso de cada task; EEVDF añade una *deadline* virtual para tratar mejor a las tasks que necesitan poca latencia.

1. Peso según `nice`. Cada nivel de `nice` cambia el peso alrededor de 1,25 veces (unos 10% de CPU). Tabla `sched_prio_to_weight` en `kernel/sched/core.c`:

nice	−20	−5	0	1	5	19
peso	88761	3121	1024	820	335	15

2. `vruntime` (tiempo de ejecución virtual). Avanza más lento cuanto mayor es el peso. Una task con nice 0 avanza al ritmo del reloj; una con más peso, más despacio:

$$\Delta vruntime = \Delta t_{real} \cdot \frac{1024}{w_i}$$

3. Tiempo virtual de la cola, `V`. Es el promedio de los `vruntime` ponderado por peso (`avg_vruntime()` en `fair.c`). Representa lo que cada task "debería" haber recibido.

4. Lag y elegibilidad. El *lag* mide cuánto servicio se le debe a una task:

$$lag_i = w_i \cdot (V - v_i)$$

Si `lag ⩾ 0` (su `vruntime` no va por delante de `V`) la task es **elegible**: ha recibido menos o lo justo. Si va adelantada, debe esperar.

5. Deadline virtual. Cada task pide un *slice* (0,7 ms × (1 + $\log_2$ CPUs); en la VM de 2 CPUs son 1,4 ms):

$$vd_i = v_i + slice \cdot \frac{1024}{w_i}$$

6. Regla de elección: entre las tasks **elegibles**, gana la de **deadline virtual más temprana**. Las tasks están en un **árbol rojo-negro** aumentado que permite encontrarla en O(log n) (`pick_eevdf()`).

Ejemplo para la expo. Dos procesos que solo calculan, en el mismo CPU, uno con nice 0 (peso 1024) y otro con nice 5 (peso 335):

- Reparto: 1024 / (1024 + 335) = **75,3%** y 335 / 1359 = **24,7%**.

- En 1 s: A corre unos 753 ms y su `vruntime` avanza 753 ms; B corre 247 ms y su `vruntime` avanza 247 × 1024 / 335 ≈ 755 ms.
- Conclusión: el tiempo **real** es distinto, pero el tiempo **virtual** queda igualado. Eso es "justo" para el scheduler.

CFS frente a EEVDF:

	CFS (2.6.23 a 6.5)	EEVDF (6.6 en adelante)
Elige	el menor `vruntime`	el elegible con menor deadline virtual
Latencia	heurísticas: `sched_latency`, `min_granularity`, `wakeup_granularity`	sale del *slice*: slice corto = deadline cercana = corre antes
Al dormir	ajustes ad hoc del `vruntime` al despertar	conserva el *lag*: no se gana ventaja durmiendo
Ajuste por task	solo `nice`	`nice` y, desde 6.12, un slice propio con `sched_setattr()`

Para profundizar: `Documentation/scheduler/sched-eevdf.rst` y el artículo original de Stoica y Abdel-Wahab (1995).

Preemption, cambio de contexto y multiprocesador

Preemption (expropiación) es quitarle la CPU a una task sin que la ceda. En Linux, marcar `TIF_NEED_RESCHED` no cambia de task en el acto: el cambio ocurre en un **punto seguro**.

- **En modo usuario:** al volver de una interrupción o de una llamada al sistema, el kernel revisa la bandera y llama a `schedule()`.
- **En modo kernel**, según cómo se compiló (`CONFIG_PREEMPT_*`):
  - `NONE`: solo cede en puntos explícitos (`cond_resched()`). Mejor rendimiento, peor latencia; típico de servidores.
  - `VOLUNTARY`: más puntos de cesión.
  - `FULL`: puede expropiar casi en cualquier punto, salvo con `preempt_disable()` o un spinlock tomado.
  - `LAZY` (desde 6.13): las tasks normales se expropian en el siguiente tick, las de tiempo real al instante.

    - `PREEMPT_RT`: tiempo real duro; integrado al kernel principal en 6.12 después de unos 20 años fuera.
- Puedes ver el modo de tu kernel con `uname -v` o en `/sys/kernel/debug/sched/preempt`.

Cambio de contexto (`context_switch()`):

1. `switch_mm()`: carga las tablas de páginas del nuevo proceso (registro CR3 en x86). Entre hilos del mismo proceso no hace falta.
2. `switch_to()`: guarda registros y puntero de pila de la task saliente y carga los de la entrante. La task "sale" de `__schedule()` donde se había quedado.
3. `finish_task_switch()`: limpieza (por ejemplo, liberar una task zombie que ya terminó).

El tick (`sched_tick()`), por defecto 1000 Hz en Fedora (`CONFIG_HZ`), actualiza estadísticas y revisa si a la task actual se le acabó el slice. Con `NO_HZ` el tick se apaga en CPUs inactivas para ahorrar energía.

Multiprocesador:

- Cada CPU tiene su runqueue, así que hace falta **balanceo de carga** (`load_balance()`): periódicamente, y cuando una CPU queda inactiva, roba tasks de la CPU más ocupada.
- El balanceo sigue **dominios de planificación** (`sched_domain`): primero entre hilos SMT del mismo núcleo, luego entre núcleos que comparten caché, luego entre nodos NUMA. Migrar lejos es más caro (se pierde la caché).
- **Afinidad:** `taskset` o `sched_setaffinity()` limitan en qué CPUs puede correr una task (`task->cpus_mask`). Lo usaremos en la demo para forzar a dos procesos a competir por la misma CPU.

sched_ext (6.12): permite cargar un scheduler completo escrito en **BPF** sin recompilar el kernel. Si falla, el kernel lo desactiva y vuelve a EEVDF. Ejemplos: `scx_simple`, `scx_rusty`, `scx_lavd` (usado por Valve para juegos). Es un buen cierre para la exposición: el scheduler ya se puede experimentar como un programa más.

KGDB

KGDB es un servidor de gdb (*stub*) dentro del propio kernel. Cuando se activa, detiene todas las CPUs y habla el protocolo remoto de gdb por un puerto serie. En otra máquina, `gdb vmlinux` se conecta y puede poner breakpoints, leer variables del kernel y avanzar paso a paso, como con un programa normal.

Por qué hacen falta dos máquinas: el kernel detenido no puede ejecutar el gdb que lo depura. En el laboratorio la "máquina depurada" es una VM de QEMU y gdb corre en Fedora.

Piezas:

- `CONFIG_KGDB`: el núcleo del depurador.
- `CONFIG_KGDB_SERIAL_CONSOLE`: el driver **kgdboc** (*KGDB over console*), que usa un puerto serie ya existente (`ttyS1`).
- `CONFIG_KGDB_KDB`: **KDB**, una consola de depuración más simple que se usa sin gdb, directamente por el puerto serie (comandos `ps`, `bt`, `md`).
- `CONFIG_DEBUG_INFO`: símbolos DWARF en `vmlinux`, para que gdb entienda tipos y nombres.
- `CONFIG_GDB_SCRIPTS`: scripts en Python para gdb (`lx-ps`, `$lx_current()`, `lx-dmesg`…).
- `nokaslr` en la línea de arranque: sin ella el kernel se carga en una dirección aleatoria y las direcciones de `vmlinux` no coinciden.

Cómo entrar al depurador:

Forma	Cuándo
`kgdboc=ttyS1,115200 kgdbwait` al arrancar	se detiene muy temprano en el arranque, esperando a gdb
`echo g > /proc/sysrq-trigger` en la VM	se detiene en cualquier momento posterior
un breakpoint que se cumple	después de `continue`, cuando el kernel pasa por esa función
un *oops* o *panic*	KGDB toma el control si está configurado

KGDB frente al gdbstub de QEMU (`qemu -s -S`): el de QEMU está *fuera* del kernel: el emulador congela la CPU virtual y no necesita ningún soporte del kernel. KGDB está *dentro*, y por eso funciona también en hardware real con un cable serie. En la VM ambos sirven; para la expo mostramos KGDB porque es lo que se usaría en una máquina física.

Limitaciones: con `CONFIG_STRICT_KERNEL_RWX` el código del kernel es de solo lectura y los breakpoints de software podrían fallar (en esta VM funcionaron bien); la alternativa son los de hardware (`hbreak`, máximo 4 en x86). Mientras KGDB tiene el control, todo el

sistema está congelado, incluidos los timers.

Comandos de gdb para la demo:

Comando	Qué hace
`target remote :1234`	conecta con KGDB en la VM
`break __sched_setscheduler`	breakpoint en una función del kernel
`continue` / `c`	deja correr el kernel
`bt`	pila de llamadas: cómo llegamos aquí
`lx-ps`	lista de tasks, como `ps`
`p $lx_current()->comm`	nombre de la task que corre en esta CPU
`p $lx_current()->se`	datos de EEVDF de esa task (`vruntime` , `deadline` , peso)
`p *p` / `p policy`	argumentos de la función donde estamos detenidos
`finish`	ejecuta hasta salir de la función actual
`lx-dmesg`	el log del kernel
`detach`	suelta el kernel y lo deja seguir

Laboratorio

El entorno ya está montado y probado en `~/Documents/expo_sistemas/lab` : kernel **7.2.7** compilado con KGDB, un initramfs con BusyBox y scripts de demo. Todos los resultados de esta sección son de una ejecución real.

Archivos:

Archivo	Para qué
`01-configurar-kernel.sh`	`defconfig` + opciones de KGDB, símbolos y `RANDOMIZE_BASE` desactivado (ya ejecutado)
`02-compilar-kernel.sh`	compila `bzImage` , `vmlinux` y los scripts de gdb (ya ejecutado, unos 3 min)

Archivo	Para qué
`03-crear-initramfs.sh`	arma el sistema de archivos con BusyBox y `/demo`
`04-arrancar-vm.sh`	arranca QEMU: consola en esta terminal, KGDB en el puerto TCP 1234
`05-conectar-gdb.sh`	abre gdb con `vmlinux` y se conecta a KGDB
`rootfs-extra/demo/*.sh`	demos que se ejecutan dentro de la VM

Qué hace cada script

En conjunto, los scripts arman dos máquinas conectadas por un cable serie: la VM con el kernel que vamos a depurar y gdb en Fedora. Del 01 al 03 se prepara el kernel y su sistema de archivos; el 04 y el 05 son las dos puntas del cable.

`01-configurar-kernel.sh` : decide con qué se compila el kernel. El kernel tiene miles de opciones guardadas en el archivo `.config` .

Paso	Qué hace y por qué
`make defconfig`	parte de la configuración por defecto para x86_64
`make kvm_guest.config`	añade los drivers para correr como invitado de una VM
`KGDB` , `KGDB_SERIAL_CONSOLE`	el depurador y su driver por puerto serie (`kgdboc`)
`KGDB_KDB` , `KDB_KEYBOARD`	la consola `kdb>` que aparece al congelar el kernel
`MAGIC_SYSRQ`	permite `echo g > /proc/sysrq-trigger` para entrar al depurador
`DEBUG_INFO_DWARF5`	guarda tipos, nombres de variables y líneas de código en `vmlinux` ; sin esto gdb solo ve direcciones
`GDB_SCRIPTS`	genera los comandos `lx-ps` , `$lx_current()` , `$lx_per_cpu()`

Paso	Qué hace y por qué
`FRAME_POINTER`, `UNWINDER_FRAME_POINTER`	hacen que `bt` muestre la pila de llamadas completa y fiable
`RANDOMIZE_BASE` desactivado	quita KASLR para que las direcciones del kernel coincidan con las de `vmlinux`
`DEBUG_FS`, `SCHEDSTATS`	más estadísticas del scheduler en `/proc/<pid>/sched` y `/sys/kernel/debug/sched`
`make olddefconfig`	resuelve las dependencias entre opciones y deja el `.config` coherente

`02-compilar-kernel.sh`: compila. Genera dos archivos del mismo kernel:

- `arch/x86/boot/bzImage` (14 MB): el kernel comprimido y listo para arrancar. Es el que carga QEMU.
- `vmlinux` (413 MB): el kernel sin comprimir y con todos los símbolos de depuración. No arranca; lo usa gdb para traducir direcciones a nombres de funciones y variables.
- `make scripts_gdb` crea `vmlinux-gdb.py`, que gdb carga para tener los comandos `lx-*`.
- `-j$(nproc)` compila en paralelo con todos los núcleos (16 en tu máquina).

`03-crear-initramfs.sh`: arma el sistema de archivos de la VM. La VM no tiene disco: todo vive en un *initramfs*, un archivo que el kernel descomprime en RAM al arrancar.

1. Crea la carpeta `rootfs/` y copia **BusyBox**, un solo ejecutable que hace de unos 300 comandos (`sh`, `ps`, `top`, `nice`, `chrt`, `taskset`…).
2. Copia `rootfs-extra/`: el programa `/init` y las demos de `/demo`.
3. Empaqueta todo con `cpio` (formato `newc`, el que entiende el kernel) y lo comprime con `gzip`, lo que da `initramfs.cpio.gz`.

El archivo **`/init`** es el primer proceso (PID 1) que ejecuta el kernel:

- `busybox --install -s /bin` crea los comandos como enlaces a BusyBox.
- Monta los sistemas de archivos virtuales: `/proc` (procesos, incluido `/proc/<pid>/sched`), `/sys`, `/dev`, `debugfs` y un `/tmp` en RAM.
- Muestra el menú de demos y abre una shell con `setsid cttyhack sh`, para que funcionen `Ctrl+C` y los trabajos en segundo plano.

`04-arrancar-vm.sh` : arranca la máquina depurada.

Opción de QEMU	Qué hace
`-enable-kvm -cpu host`	usa la virtualización del procesador; la VM corre casi a velocidad nativa
`-smp 2 -m 1G`	2 CPUs (para que haya dos runqueues) y 1 GB de RAM
`-kernel bzImage -initrd initramfs.cpio.gz`	arranca directamente el kernel y el initramfs, sin disco ni GRUB
`-append "..."`	la línea de comandos del kernel (abajo)
primer `-serial mon:stdio`	puerto `ttyS0` = tu terminal: ahí ves la consola. `Ctrl+A X` sale
segundo `-serial tcp:127.0.0.1:1234`	puerto `ttyS1` = el "cable" de KGDB, expuesto como puerto TCP 1234
`-display none -no-reboot`	sin ventana gráfica; si el kernel falla, QEMU se cierra en vez de reiniciar

La línea de comandos del kernel: `console=ttyS0` (los mensajes van al primer puerto), `nokaslr` (direcciones fijas), `kgdboc=ttyS1,115200` (KGDB escucha en el segundo puerto) y, con `--wait` , `kgdbwait` (se detiene al arrancar hasta que gdb se conecte).

`05-conectar-gdb.sh` : la punta del depurador.

- `gdb vmlinux` : abre el kernel con símbolos, como se haría con cualquier programa.
- `add-auto-load-safe-path` : autoriza a gdb a cargar `vmlinux-gdb.py` (por seguridad no carga scripts sin permiso).
- `target remote 127.0.0.1:1234` : se conecta al puerto serie de la VM, donde KGDB espera.

Las demos (`rootfs-extra/demo/`) son scripts de shell que se ejecutan dentro de la VM:

- `nice.sh` : `taskset -c 0` fija dos bucles infinitos en la CPU 0 para que compitan, `nice -n 0` y `nice -n 5` les dan distinto peso, y luego lee `top` y `/proc/<pid>/sched` .
- `rt.sh` : lanza `sleep 30` y lo pasa a tiempo real con `chrt -f -p 50` , que hace la llamada al sistema `sched_setscheduler` .

- `fork.sh`: `sh -c` crea un proceso hijo, que pasa por `fork` → `kernel_clone` → `wake_up_new_task`.

Arranque (dos terminales):

1. Terminal 1: `./04-arrancar-vm.sh`. Aparece el prompt `/ #` de la VM.
2. En la VM: `echo g > /proc/sysrq-trigger`. El kernel se congela y muestra `[1]kdb>` (la consola KDB).
3. Terminal 2: `./05-conectar-gdb.sh`. gdb se conecta y muestra `kgdb_breakpoint ()`.
4. Para soltar la VM: `continue` (o `detach` para terminar). Para salir de QEMU: `Ctrl+A` y luego `X`.

Regla de oro: gdb solo acepta comandos cuando el kernel está **detenido** (aparece el prompt `(gdb)`). Después de `continue`, gdb muestra `Continuing.` y lo que escribas no se ejecuta: queda en espera y se ejecutará más tarde, posiblemente donde no quieres. Para poner un breakpoint nuevo:

1. En la VM: `echo g > /proc/sysrq-trigger`. gdb muestra `received signal SIGTRAP` y vuelve el prompt `(gdb)`.
2. En gdb: `break <función>` (un comando por línea, sin comas).
3. En gdb: `continue`.
4. En la VM: ejecuta la demo. Ahora sí se detiene en el breakpoint.

Usa `break`, no `hbreak`: en esta VM con KVM los breakpoints de hardware se aceptaron pero no se dispararon; los de software funcionan.

Experimento 1: reparto de CPU según nice (sin gdb)

En la VM: `/demo/nice.sh`. Lanza dos bucles infinitos en la CPU 0, con nice 0 y nice 5, y a los 5 s muestra `top` y `/proc/<pid>/sched`.

	A (nice 0)	B (nice 5)
`prio`	120	125
`se.load.weight`	1048576 (= 1024 × 1024)	343040 (= 335 × 1024)
`se.sum_exec_runtime` (tiempo real)	3861 ms (75,3%)	1266 ms (24,7%)
`se.vruntime` (tiempo virtual)	4296 ms	4304 ms
`top` (% del total de 2 CPUs)	40%	10%

Qué señalar: el reparto real coincide con la cuenta de la sección EEVDF (75,3% / 24,7%) y los `vruntime` quedan casi iguales. El peso aparece multiplicado por 1024 porque el kernel lo guarda con más precisión. `top` muestra 40% y 10% porque divide entre las 2 CPUs. Al terminar: `kill` con los PID que imprime el script.

Experimento 2: un proceso pasa a tiempo real

En gdb:

```
break __sched_setscheduler
continue
```

En la VM: `/demo/rt.sh` (ejecuta `chrt -f -p 50` sobre un `sleep`). La VM se congela y gdb se detiene. Luego:

```
bt                  # syscall sched_setscheduler -> do_sched_setscheduler ->
__sched_setscheduler
p p->comm           # "sleep"
p p->policy         # 0 = SCHED_NORMAL
p p->prio           # 120
p *attr             # sched_policy = 1 (FIFO), sched_priority = 50
finish
p p->policy         # 1 = SCHED_FIFO
p p->prio           # 49 (= 99 - 50)
p p->sched_class    # <rt_sched_class>
```

Qué señalar: la `bt` muestra el camino desde la llamada al sistema hasta el scheduler, y la task cambia de clase (`fair` → `rt`) y de prioridad interna (120 → 49). Termina con `delete` y `continue`.

Experimento 3: nace un proceso

Con el kernel detenido (prompt `(gdb)`), escribe en gdb, un comando por línea:

```
break wake_up_new_task
continue
```

En la VM: `/demo/fork.sh`. Al detenerse:

```
bt                                  # fork -> kernel_clone ->
wake_up_new_task
p p->pid                            # PID del hijo nuevo
p p->se.vruntime                    # 0: aún no está en la cola
p $lx_current()->comm               # "sh": el padre está en la CPU
p $lx_per_cpu(runqueues).nr_running  # tasks listas en esta CPU
p $lx_per_cpu(runqueues).nr_switches # cambios de contexto desde el arranque
finish
p p->se.vruntime                    # 307982277: colocado cerca del V de la
cola
p p->se.deadline                    # 308682277 = vruntime + 0,7 ms
p p->se.slice                       # 1400000 (1,4 ms)
p p->sched_class                    # <fair_sched_class>
```

Qué señalar: la task nueva no empieza con `vruntime` 0 (eso le daría ventaja injusta): EEVDF la coloca cerca del promedio `V`. Su primera deadline usa **medio slice** (0,7 de 1,4 ms), para que arranque pronto sin perjudicar a las demás (`PLACE_DEADLINE_INITIAL` en `fair.c`).

Si algo falla en vivo

- gdb no conecta: confirma que la VM está en `kdb>` (después de `echo g > /proc/sysrq-trigger`) antes de abrir gdb.
- La primera conexión tarda unos segundos mientras carga los símbolos de `vmlinux` (413 MB). Es normal.
- El mensaje `NMI handler (kgdb_nmi_handler) took too long` solo indica que el kernel estuvo congelado. Es normal.
- Ten capturas de los tres experimentos por si la demo falla.

Guion de la parte práctica

Unos **8 minutos** en 7 pasos. En cada paso: **Haces** es lo que escribes; **Dices** es lo que explicas mientras tanto. No lo memorices palabra por palabra: aprende la idea de cada "Dices" y los números clave. Los números son los de tu ensayo; en vivo saldrán parecidos, así que lee los que aparezcan en pantalla.

Antes de empezar (no se dice): dos terminales lado a lado en `~/Documents/expo_sistemas/lab`, letra grande, la VM cerrada y las capturas abiertas en otra ventana por si acaso.

Paso 0: el montaje (1 min)

Haces: en la terminal izquierda, `./04-arrancar-vm.sh`.

Dices: "Para ver el scheduler por dentro hay que depurar el kernel mientras corre. Pero si congelo el kernel de mi propia computadora, se congela todo, incluido el depurador. Por eso uso dos máquinas: a la izquierda, una máquina virtual con un kernel 7.2.7 que compilé con KGDB; a la derecha, gdb en mi Fedora. Las une un puerto serie virtual, que hace de cable. La VM no tiene disco: arranca solo el kernel y un sistema mínimo con BusyBox, por eso tarda menos de un segundo."

Paso 1: el reparto según nice (2 min)

Haces: en la VM, `/demo/nice.sh`.

Dices (mientras espera 5 s): "Primero, sin depurador. Lanzo dos procesos que solo hacen un bucle infinito, los dos obligados a usar la CPU 0 para que compitan. Uno tiene nice 0 y el otro nice 5."

Dices (con el resultado en pantalla), señalando cada línea:

- "`sum_exec_runtime` es el tiempo real de CPU: **3856 ms** contra **1264 ms**. Eso es **75% y 25%**, exactamente lo que da la cuenta con los pesos: 1024 entre 1024 más 335."
- "Pero el `vruntime`, el tiempo virtual, está casi igual: **4100 y 4107**. Esa es la idea del scheduler: el tiempo real es desigual a propósito, y lo que se iguala es el tiempo virtual. Para EEVDF eso es ser justo."
- "El peso aparece multiplicado por 1024, 1048576 y 343040, porque el kernel lo guarda con más precisión. Y `top` dice 38% y 9% porque reparte entre las dos CPUs de la VM."

Haces: `kill 73 74` (los PID que imprime el script).

Paso 2: congelar el kernel y conectar gdb (1 min)

Haces: en la VM, `echo g > /proc/sysrq-trigger`.

Dices: "Ahora congelo el kernel. Aparece `kdb`, la consola del depurador del kernel: la VM ya no responde."

Haces: en la terminal derecha, `./05-conectar-gdb.sh`.

Dices: "gdb carga los símbolos de `vmlinux`, el kernel sin comprimir, y me dice que estamos detenidos en `kgdb_breakpoint`. Desde aquí puedo leer cualquier estructura del kernel."

Haces (opcional, si vas bien de tiempo): `lx-ps`.

Dices: "Esta es la lista de tasks leída directamente de la memoria del kernel: cada una es una `task_struct`, el PCB de Linux."

Paso 3: un proceso pasa a tiempo real (2 min)

Haces: en gdb, `break __sched_setscheduler` y luego `continue`.

Dices: "Pongo un breakpoint en la función del kernel que cambia la política de planificación y dejo correr el kernel."

Haces: en la VM, `/demo/rt.sh`.

Dices: "Este script usa `chrt` para pasar un proceso `sleep` a tiempo real, FIFO con prioridad 50. Y ahí está: la VM se congeló dentro de la función."

Haces y dices, comando por comando:

Haces (en gdb)	Ves	Dices
`bt`	`__sched_setscheduler` ← `sched_setscheduler` ← syscall	"Este es el camino: la llamada al sistema `sched_setscheduler` entra al kernel y llega a `__sched_setscheduler`."
`p p->comm`	`"sleep"`	"Es el proceso `sleep`."
`p p->policy` y `p p->prio`	`0` y `120`	"Política 0, la normal, y prioridad interna 120, que es nice 0."
`p *attr`	`sched_policy = 1`, `sched_priority = 50`	"Esto es lo que pidió: política 1, FIFO, prioridad 50."

Haces (en gdb)	Ves	Dices
`finish`	`Value returned is $5 = 0`	"Dejo que la función termine. Devolvió 0: el cambio tuvo éxito."
`p p->policy` y `p p->prio`	`1` y `49`	"Ahora es FIFO, con prioridad interna 49: 99 menos 50. En el kernel, menor es más prioritario."
`p p->sched_class`	`<rt_sched_class>`	"Y cambió de clase: ahora está en `rt_sched_class`. Desde este momento le gana a cualquier proceso normal."
`delete` (responde `y`) y `continue`	`Continuing.`	"Quito el breakpoint y suelto el kernel." En la VM aparece el "Después" de `rt.sh` con SCHED_FIFO.

Paso 4: nace un proceso (1,5 min)

Haces: en la VM, `echo g > /proc/sysrq-trigger` (gdb recupera el control). En gdb, `break wake_up_new_task` y luego `continue`. En la VM, `/demo/fork.sh`.

Dices: "Ahora veamos nacer un proceso. Pongo el breakpoint en `wake_up_new_task`, donde el hijo recién creado entra a la cola del scheduler."

Haces (en gdb)	Ves	Dices
`p p->pid`	el mismo número que la VM muestra como `Padre PID` (84 en tu ensayo)	"El primer proceso que nace es el que ejecuta el script: la shell hace `fork` para lanzarlo."
`p p->se.vruntime`	`0`	"Su `vruntime` es 0: todavía no está en la cola."
`finish`	vuelve a `kernel_clone`	"Dejo que entre a la cola."

Haces (en gdb)	Ves	Dices
`p p->se.vruntime`	un número grande (34709667557 en tu ensayo)	"Ya no es 0. Si lo dejara en 0, iría tan atrasado que acapararía la CPU. El scheduler lo coloca cerca del promedio de la cola: unos 34,7 segundos de tiempo virtual."
`p p->se.deadline`	`vruntime + 700000`	"Su deadline virtual es el `vruntime` más 0,7 ms: medio slice. Así el proceso nuevo arranca rápido sin perjudicar a nadie."
`p p->se.slice`	`1400000`	"El slice completo es 1,4 ms: 0,7 ms base por 2, porque la VM tiene 2 CPUs."
`p p->sched_class`	`<fair_sched_class>`	"Es un proceso normal: clase fair, o sea EEVDF."
`delete` (responde `y`) y `continue`	`Continuing.`	"Suelto el kernel." En la VM aparecen `Padre PID` e `Hijo PID`.

Paso 5: cierre (30 s)

Dices: "En resumen: lo que vimos en teoría (PCB, cola de listos, políticas) está en Linux como `task_struct`, runqueues y clases de planificación. Con los números vimos que EEVDF iguala el tiempo virtual, no el real, y con KGDB lo vimos todo en memoria mientras el kernel corría."

Paso 6: cerrar (sin hablar)

1. En la VM: `Ctrl+A`, suelta, y `X`. QEMU se cierra.
2. En gdb aparece que se perdió la conexión y vuelve `(gdb)`: escribe `quit`.

En ese orden: mientras gdb dice `Continuing.` no acepta `quit`.

Mensajes normales que verás (por si preguntan)

Mensaje	Qué significa
`[New Thread 81]`, `[Thread 71 exited]`	gdb trata cada task del kernel como un hilo; avisa cuando nacen y mueren procesos
`Thread 62 hit Breakpoint 1`	62 es el número que gdb le da al hilo, no el PID
`received signal SIGTRAP`	entraste al depurador con `echo g`; es lo esperado
`NMI handler (kgdb_nmi_handler) took too long`	el kernel estuvo congelado ese tiempo
`clocksource: Watchdog ... timed out`	el reloj se queja por el congelamiento; no afecta
`\000\000...` después de `"sleep"`	`comm` es un arreglo de 16 bytes; el resto está vacío

Si algo sale mal en vivo

- **El breakpoint no se detiene:** seguramente escribiste el `break` después de `continue`. Haz `echo g > /proc/sysrq-trigger` en la VM y vuelve a ponerlo.
- **Salen mensajes `clocksource` o `TSC unstable`:** normales. El reloj se queja porque el kernel estuvo congelado. Si preguntan: "es el efecto de haber congelado el kernel".
- **Se cuelga todo:** en la VM `Ctrl+A` y luego `X` para cerrar QEMU, y sigue con las capturas: "para no perder tiempo, les muestro el resultado que obtuve en mi ensayo".

Preguntas probables

¿Cuál es la diferencia entre proceso e hilo en Linux? Para el kernel ambos son una `task_struct`. Un hilo es una task creada con `clone()` que comparte `mm` (memoria), archivos y señales con otra. Todas las tasks de un proceso tienen el mismo `tgid`.

¿Por qué no hay estado "listo" separado de "ejecutando"? Ambos son `TASK_RUNNING`. La que está ejecutándose es `rq->curr`; las demás esperan en la runqueue.

¿El scheduler de Linux es Round Robin, por prioridades o qué? Depende de la clase: `SCHED_RR` es Round Robin con prioridades, `SCHED_FIFO` es por prioridad sin quantum, `SCHED_DEADLINE` es EDF, y las tasks normales usan EEVDF, que es un algoritmo de reparto proporcional (*fair share*).

¿Qué problema traía CFS que resuelve EEVDF? CFS solo sabía repartir; la latencia dependía de varias heurísticas difíciles de ajustar. EEVDF añade la deadline virtual: una task que pide slices cortos (interactiva) recibe la CPU antes, sin recibir más CPU en total.

¿Qué significa nice? Es un ajuste de −20 a +19 que cambia el peso de la task. Cada punto equivale a un 10% de CPU aprox. Un usuario normal solo puede subirlo (ser "más amable"); bajarlo requiere privilegios.

¿Puede una task de tiempo real congelar el sistema? Casi: siempre gana a las normales. Por eso el kernel reserva tiempo para la clase fair (RT throttling o *deadline server*).

¿Cómo evita Linux la inanición (*starvation*) entre tasks normales? Con el `vruntime`: quien menos ha corrido queda atrás en tiempo virtual, se vuelve elegible y termina eligiéndose. Incluso nice 19 recibe algo (peso 15 frente a 1024).

¿Qué es el cambio de contexto y cuánto cuesta? Guardar el estado de una task y cargar el de otra (`context_switch()`): registros, pila y tablas de páginas. Cuesta del orden de microsegundos, más la pérdida de caché y TLB.

¿Por qué una runqueue por CPU y no una global? Una cola global obligaría a todas las CPUs a pelear por un candado. Con una por CPU cada una decide sola y se mantiene la caché; el costo es tener que balancear la carga.

¿Para qué sirve KGDB si existe printk? `printk` obliga a recompilar para cada pregunta. KGDB permite inspeccionar cualquier estructura en vivo, ver la pila de llamadas y detener el kernel justo donde ocurre algo, incluso después de un fallo.

¿Por qué usaron una máquina virtual? KGDB congela el kernel entero; si fuera el de la propia máquina no habría nadie para ejecutar gdb. La VM hace de segunda máquina y el puerto serie virtual hace de cable.

¿Qué es sched_ext? Una clase de planificación (6.12) que delega las decisiones en un programa BPF cargado en caliente. Si el programa se equivoca, el kernel vuelve a EEVDF.

Checklist de repaso y glosario

Antes de la exposición, tienes que poder:

- [ ] Explicar qué es `task_struct` y nombrar 5 de sus campos.
- [ ] Relacionar los estados de `ps` (R, S, D, T, Z) con el diagrama de estados de clase.
- [ ] Recorrer el camino de `fork()` hasta `wake_up_new_task()`.
- [ ] Nombrar las 6 clases de planificación en orden y una política de cada una.
- [ ] Explicar el ciclo `TIF_NEED_RESCHED` → `__schedule()` → `pick_next_task()` → `context_switch()`.
- [ ] Definir `vruntime`, lag, elegibilidad y deadline virtual sin mirar.
- [ ] Hacer en la pizarra el cálculo nice 0 frente a nice 5 (75% / 25%).
- [ ] Decir en una frase qué cambió de CFS a EEVDF.
- [ ] Explicar qué es KGDB, por qué necesita otra máquina y para qué sirve `nokaslr`.
- [ ] Correr la demo completa de principio a fin al menos dos veces.
- [ ] Tener capturas de cada paso de la demo como respaldo.

Glosario:

Término	Significado
task	unidad que planifica Linux; proceso o hilo
runqueue (`struct rq`)	cola de tasks listas de una CPU
clase de planificación	módulo con su propio algoritmo (stop, dl, rt, fair, ext, idle)
peso	número derivado de `nice`; nice 0 = 1024
`vruntime`	tiempo de CPU recibido, dividido por el peso relativo
lag	diferencia entre lo que debía recibir una task y lo que recibió
elegible	task con lag ≥ 0; puede ser elegida
deadline virtual	`vruntime` + slice ponderado; el menor entre los elegibles gana
slice	porción de CPU que pide una task antes de volver a competir
preemption	quitar la CPU a una task sin que la ceda

Término	Significado
`TIF_NEED_RESCHED`	bandera que pide replanificar en el siguiente punto seguro
balanceo de carga	mover tasks entre runqueues de distintas CPUs
afinidad	conjunto de CPUs donde puede correr una task
KGDB	servidor de gdb dentro del kernel
kgdboc	KGDB por puerto serie o consola
KASLR	carga del kernel en dirección aleatoria; se desactiva con `nokaslr`
initramfs	sistema de archivos mínimo que el kernel carga en memoria al arrancar

Fuentes: docs.kernel.org: EEVDF · docs.kernel.org: KGDB · docs.kernel.org: gdb y scripts del kernel · docs.kernel.org: clases de planificación · Elixir: código del kernel · LWN.net, busca "EEVDF" y "sched_ext".